



Lab Project Report

Compiler Construction Lab (CSE 0613 3330)

Submitted To:

Khalid Bin Selim

Lecturer

Department of Computer Science & Engineering
Metropolitan University

Submitted By:

Rajib Dab 231-115-103

Md. Monsur Alam 231-115-104

August 1, 2026

Mini Programming Language Compiler

Compiler Construction Lab

Metropolitan University, Bangladesh

Team: Razibit and The Tokenizers

Rajib Dab (231-115-103) Md Monsur Alam (231-115-104)

August 1, 2026

Abstract

This project implements a complete compiler front end for a small, statically typed programming language. A Flex lexer converts source characters into tokens, a Bison LALR parser with one token of lookahead validates grammar and constructs an abstract syntax tree (AST), and C routines maintain scoped symbols, perform semantic checks, and generate three-address code (TAC). A Flask-based GUI is included as a bonus demonstration surface; the compiler itself remains a command-line Flex/Bison/C program.

1 Objectives and Architecture

The objective is to recognize source text, validate structure, preserve meaning in a tree, track declarations and scopes, reject invalid programs, and produce TAC.



Figure 1: Compiler front-end pipeline.

The parser builds the AST bottom-up during reductions. After parsing, the `gen_code` function recursively walks the AST, checking declarations and types while emitting TAC. Semantic analysis and TAC generation are deliberately combined in this traversal.

2 Language and Technologies

Table 1: Supported language features and implementation tools.

Area	Supported features
Types	<code>int</code> , <code>float</code> , <code>bool</code>
Statements	Declaration, assignment, <code>if</code> , <code>if-else</code> , <code>while</code> , <code>print</code> , and blocks
Expressions	Arithmetic, relational, equality, logical, and unary operators
Tools	Flex, Bison, GCC, Make, and C
Bonus	Flask and a terminal-style HTML/CSS/JavaScript GUI

3 Lexical Analysis

The file `src/lexer/lexer.l` recognizes keywords, identifiers, literals, operators, delimiters, comments, and invalid characters. Keyword rules precede the identifier rule, longest-match behavior handles operators such as `<=`, `%option yylineno` tracks lines, and the catch-all rule reports unknown characters.

```
int x;  
x = 10;
```

Representative output:

```
KEYWORD int  
IDENTIFIER x  
SEMICOLON ;  
IDENTIFIER x  
ASSIGN =  
INT_LITERAL 10  
SEMICOLON ;
```

The lexer recognizes spelling and boundaries; declaration and type correctness belong to later phases.

4 Syntax Analysis and AST

The Bison grammar uses an LALR parser with one token of lookahead and precedence declarations ordered from logical operators through equality, relational, additive, multiplicative, and unary operators. A conceptual parse tree includes punctuation and grammar wrappers; the compiler prints the more compact AST.

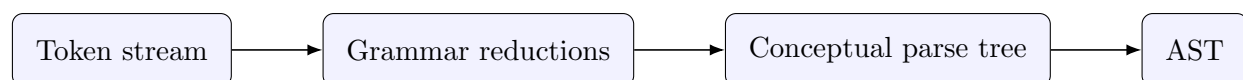


Figure 2: Relationship between tokens, reductions, and the AST.

Each AST node stores its node kind, inferred data type, operator or name, literal value, source line, and up to three children.

```
Assign [y =]  
  BinaryOp [+]  
    Identifier [y]  
    Identifier [x]
```

5 Symbol Table and Semantic Analysis

The symbol table is a stack of fixed-size scope arrays. Each entry stores a name, type, declaration line, and usage flag. Declarations enter the current scope; duplicate names in that scope are rejected; lookup searches from the innermost scope outward.

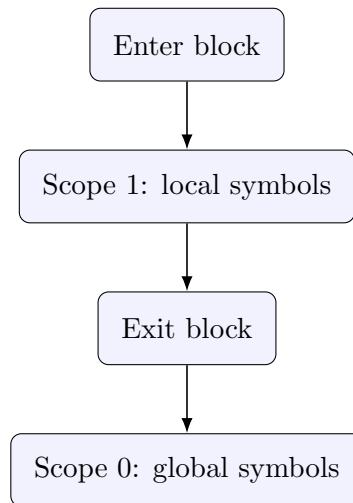


Figure 3: Nested-scope lifecycle.

During the `gen_code` traversal, identifiers are looked up and expression types are inferred. The implementation detects undeclared variables, same-scope redeclarations, incompatible assignments, invalid binary operands, invalid logical operands, and applying `!` to a non-Boolean expression.

```
int x;  
x = 3.14;
```

This example is syntactically valid but semantically invalid because the right-hand side is a floating-point value and `x` requires an integer.

6 Three-Address Code

TAC decomposes expressions into simple instructions. Temporaries use `t1`, `t2`, ..., and control-flow labels use `L0`, `L1`,

```
t1 = y + x
y = t1
t2 = x > 0
if_false t2 goto L1
goto L0
L1:
```

Loops receive a condition temporary, false exit label, body, back-edge, and exit label. This representation is a useful bridge toward optimization or target-code generation.

7 Workflow, GUI, and Testing

1. `make` runs Flex, Bison, and GCC.
2. `./compiler examples/valid/sample.mc` opens the source file.
3. The `yyparse` function requests tokens from Flex and performs reductions.
4. Grammar actions construct `ast_root`.
5. The AST and symbol table are printed.
6. The `gen_code` function checks semantics and emits TAC.
7. The summary reports success or failure.

The GUI sends source text to Flask, which runs the same compiler binary and separates labeled output into Tokens, AST, Symbol Table, TAC, Errors, and Summary. It does not replace a compiler phase.

Table 2: Test-suite coverage.

Test group	Coverage
tests/valid/	Arithmetic, loops, conditionals, and nested scopes
tests/invalid/	Lexical, syntax, undeclared-variable, redeclaration, and type errors
examples/valid/sample.mc	Primary presentation demonstration

8 Challenges, Limitations, and Future Work

Flex/Bison setup on Windows required WSL or Linux. Precedence declarations resolve expression ambiguity. Keeping most logic in `parser.y` simplifies demonstration but reduces modularity. The project stops at TAC, uses fixed-size arrays, and does not support functions, arrays, optimization, or machine code. Future work includes separate passes, richer recovery, constant folding, dead-code elimination, and a target backend.

9 Conclusion

The project demonstrates a traceable path from characters to intermediate representation: the lexer supplies tokens, the parser builds structure, the AST preserves meaning, the symbol table supplies context, semantic analysis validates meaning, and TAC exposes control flow.

10 Screenshots and Repository

The following captures were produced from the running GUI and the actual compiler binary. They are stored in the `project` directory and are included here as evidence of the implemented pipeline and its output.

mini-compiler - razibit@tokenizers:~/project

[MINI COMPILER]
Razibit and The Tokenizers · Metropolitan University, Bangladesh
● SUCCESS

SOURCE CODE .mc
CLR
SAMPLE

```

1  /* Sample program - demonstrates all features */
2  int x;
3  int y;
4  bool flag;
5
6  x = 10;
7  y = 0;
8  flag = true;
9
10 while (x > 0) {
11     y = y + x;
12     x = x - 1;
13 }
14
15
16 if (flag == true) {
17     print y;
18 } else {
19     print x;

```

COMPILER OUTPUT

TOKENS
AST
SYM TABLE
TAC
ERRORS

```

; declare int x
; declare int y
; declare bool flag
x = 10
y = 0
flag = true
L0:
t1 = x > 0
if_false t1 goto L1
t2 = y + x
y = t2
t3 = x - 1
x = t3
goto L0
L1:
t4 = flag == true
if_false t4 goto L2

```

Ctrl+Enter to compile
\$ COMPILE

✓ Compilation successful - Three Address Code generated.

COMPILER PIPELINE

01
LEXER
Flex
Tokenizes source

02
PARSER
Bison
Validates grammar

03
AST
C Structs
Builds syntax tree

04
SYM TABLE
Scoped Stack
Tracks variables

05
SEMANTIC
Type checker
Validates meaning

06
TAC GEN
Three Address
Code output

Compiler Construction Lab · Metropolitan University, Bangladesh · 2026

Figure 4: GUI editor and compiler pipeline.

mini-compiler - razibit@tokenizers:~/project

[MINI COMPILER] Razibit and The Tokenizers · Metropolitan University, Bangladesh

SUCCESS

SOURCE CODE .mc

CLR

SAMPLE

```
1  /* Sample program - demonstrates all features */
2  int x;
3  int y;
4  bool flag;
5
6  x = 10;
7  y = 0;
8  flag = true;
9
10 while (x > 0) {
11     y = y + x;
12     x = x - 1;
13 }
14
15 if (flag == true) {
16     print y;
17 } else {
18     print x;
19 }
```

Ctrl+Enter to compile

\$ COMPILE

COMPILER OUTPUT

TOKENS

AST

SYM TABLE

TAC

ERRORS

[Line 2]	KEYWORD	int
[Line 2]	IDENTIFIER	x
[Line 2]	SEMICOLON	;
[Line 3]	KEYWORD	int
[Line 3]	IDENTIFIER	y
[Line 3]	SEMICOLON	;
[Line 4]	KEYWORD	bool
[Line 4]	IDENTIFIER	flag
[Line 4]	SEMICOLON	;
[Line 6]	IDENTIFIER	x
[Line 6]	ASSIGN	=
[Line 6]	INT_LIT	10
[Line 6]	SEMICOLON	;
[Line 7]	IDENTIFIER	y
[Line 7]	ASSIGN	=
[Line 7]	INT_LIT	0
[Line 7]	SEMICOLON	;

✓ Compilation successful - Three Address Code generated.

COMPILER PIPELINE

01
LEXER
Flex
Tokenizes source

→

02
PARSER
Bison
Validates grammar

→

03
AST
C Structs
Builds syntax tree

→

04
SYM TABLE
Scoped Stack
Tracks variables

→

05
SEMANTIC
Type checker
Validates meaning

→

06
TAC GEN
Three Address
Code output

Compiler Construction Lab · Metropolitan University, Bangladesh · 2026

Figure 5: GUI token output.

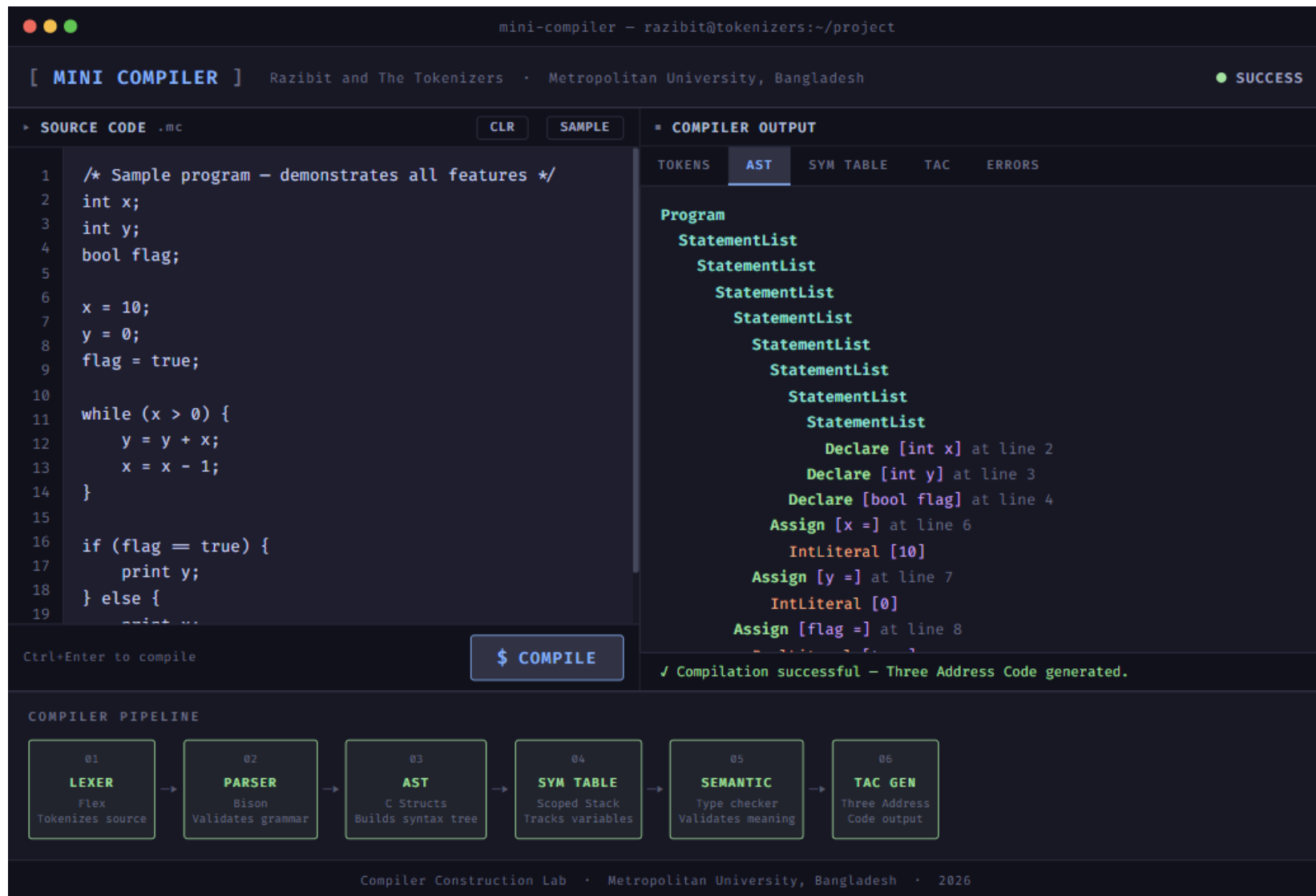


Figure 6: GUI AST output.

mini-compiler - razibit@tokenizers:~/project

[**MINI COMPILER**] Razibit and The Tokenizers · Metropolitan University, Bangladesh ● SUCCESS

► **SOURCE CODE** .mc CLR SAMPLE

```

1  /* Sample program - demonstrates all features */
2  int x;
3  int y;
4  bool flag;
5
6  x = 10;
7  y = 0;
8  flag = true;
9
10 while (x > 0) {
11     y = y + x;
12     x = x - 1;
13 }
14
15
16 if (flag == true) {
17     print y;
18 } else {
19     print x;

```

Ctrl+Enter to compile \$ **COMPILE**

COMPILER OUTPUT

TOKENS AST **SYM TABLE** TAC ERRORS

Scope 0:
(no variables)

[Performing semantic analysis and TAC generation ...]

✓ **Compilation successful - Three Address Code generated.**

COMPILER PIPELINE

```

graph LR
    01[01 LEXER  
Flex  
Tokenizes source] --> 02[02 PARSER  
Bison  
Validates grammar]
    02 --> 03[03 AST  
C Structs  
Builds syntax tree]
    03 --> 04[04 SYM TABLE  
Scoped Stack  
Tracks variables]
    04 --> 05[05 SEMANTIC  
Type checker  
Validates meaning]
    05 --> 06[06 TAC GEN  
Three Address  
Code output]

```

Compiler Construction Lab · Metropolitan University, Bangladesh · 2026

Figure 7: GUI symbol-table output.

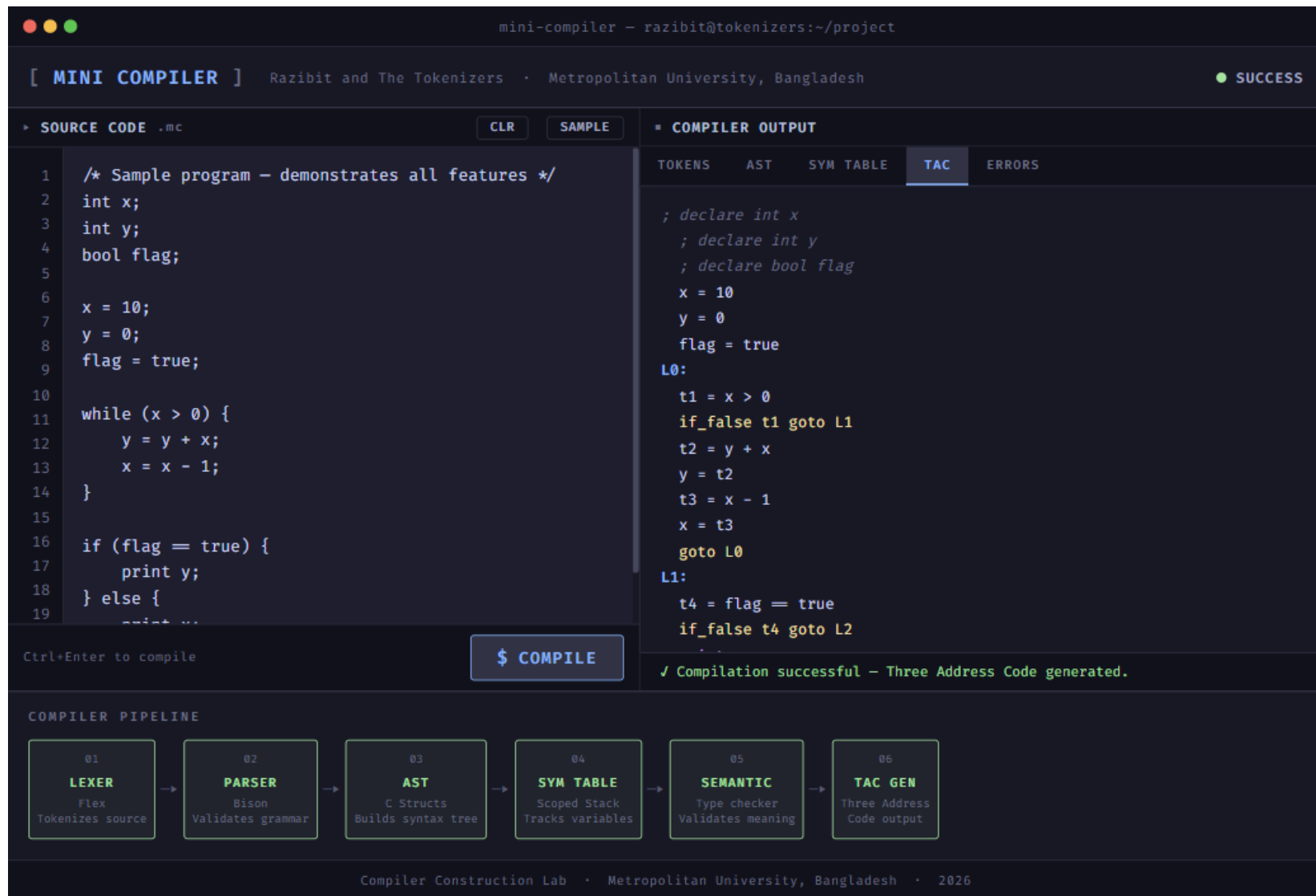


Figure 8: GUI TAC output.

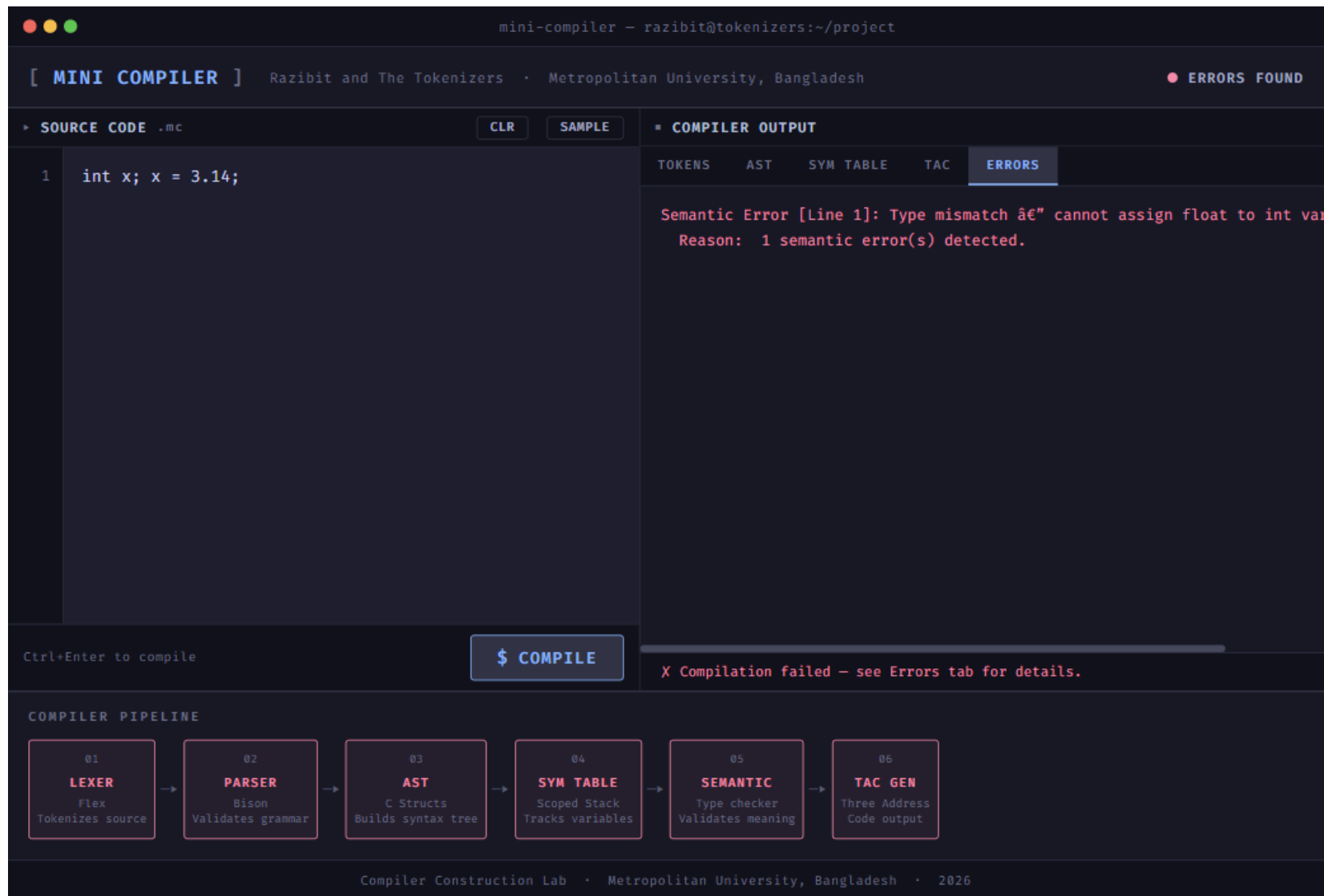


Figure 9: GUI semantic-error output.

```
razibit@tokenizers:~/project$ ./compiler examples/valid/sample.mc
```

```
Mini Compiler – Razibit and The Tokenizers
```

```
===== TOKENS =====
```

```
[Line 7] KEYWORD int [Line 7] IDENTIFIER x
```

```
[Line 13] IDENTIFIER x ASSIGN = INT_LIT 10
```

```
===== ABSTRACT SYNTAX TREE =====
```

```
Program
```

```
Declare [int x] Assign [x =]
```

```
While → Condition: BinaryOp [>]
```

```
===== SYMBOL TABLE =====
```

```
Scope 0: x int, y int, flag bool, rate float
```

```
===== THREE ADDRESS CODE =====
```

```
t1 = x > 0 if_false t1 goto L1 t2 = y + x
```

```
y = t2 t3 = x - 1 x = t3 goto L0 L1:
```

```
===== COMPILATION SUMMARY =====
```

```
Result: SUCCESS – Three Address Code generated successfully.
```

Figure 10: Successful terminal compilation.

```
razibit@tokenizers:~/project$ ./compiler tests/invalid/test_type_mismatch.mc
```

```
Semantic Error [Line 10]: cannot assign float to int variable 'x'.
```

```
Semantic Error [Line 13]: cannot assign int to bool variable 'flag'.
```

```
Semantic Error [Line 16]: cannot assign bool to float variable 'rate'.
```

```
===== THREE ADDRESS CODE =====
```

```
x = 3.1400 t1 = 3 + 3 flag = t1 rate = true
```

```
===== COMPILATION SUMMARY =====
```

```
Result: FAILED
```

```
Reason: 3 semantic error(s) detected.
```

Figure 11: Terminal semantic-error compilation.

Repository: GitHub project page.

References

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Addison-Wesley, 2006.
- [2] The Flex Project, *Flex: The Fast Lexical Analyzer*, <https://westes.github.io/flex/manual/>.
- [3] Free Software Foundation, *GNU Bison Manual*, <https://www.gnu.org/software/bison/manual/>.
- [4] *Compiler Construction Lab Project Manual.pdf*, course project manual.